

Universidade de Aveiro – Dep. de Eletrónica, Telecomunicações e Informática
Laboratório de Sistemas Digitais – Ano Letivo 2021/22 – Exame de Época Recurso

150 - RECURSO
18/07/2022

Notas Importantes:

1. Verifique, para todas as questões, qual a resposta correta e assinale com um "X" a sua escolha na tabela ao lado. Por cada resposta incorreta será descontada, à cotação global, 1/3 da cotação da respetiva pergunta.
2. Durante a realização do teste não é permitida a permanência junto do aluno, mesmo que desligado, de qualquer dispositivo eletrónico não expressamente autorizado (nesta lista incluem-se calculadoras, telemóveis, smartwatches e qualquer outro dispositivo de captura de imagem e/ou comunicação). A sua deteção durante a realização do exame implica a imediata anulação do mesmo.
3. Não é permitido escrever na área branca em torno da matriz de respostas.
4. Cotações: Grupo I (24 perguntas) – cada 0.5 valores; Grupo II (8 perguntas) – cada 1 valor

Grupo I

1. A O processo VHDL seguinte define um sinal cuja frequência e o duty-cycle são, respetivamente:

```

clock_proc: process
begin
    s_clk <= '0'; wait for 40 ns;
    s_clk <= '1'; wait for 60 ns;
end process;

```

 - a) 10 MHz e 60%
 - b) 12.5 MHz e 50%
 - c) 10 KHz e 60%
 - d) 10 MHz e 40%
2. A partir do extrato de código seguinte, determine o tipo do sinal sig.

```

signal s_a, s_b : signed(3 downto 0);
...
s_a <= "1011";
s_b <= "1111";
sig <= s_a - s_b;

```

 - a) signed(3 downto 0)
 - b) signed(7 downto 0)
 - c) std_logic_vector(3 downto 0)
 - d) é impossível determinar inequivocamente o tipo de sig
3. Continuando a análise do código da pergunta 2, determine o resultado de execução da linha seguinte:

```
to_integer(s_a / s_b);
```

 - a) 0
 - b) 5
 - c) -5
 - d) 1
4. A lookup table (tabela de verdade) da figura ao lado implementa a seguinte função lógica:
 - a) $z \leq a \text{ and } b$;
 - b) $z \leq a \text{ xor } b$;
 - c) $z \leq a \text{ or } b$;
 - d) $z \leq a \text{ nor } b$;

Ligação de I/O		Posição (12, 11, 10)	Out = f(12, 11)
a	10	000	0
b	11	001	0
		010	0
		011	1
		100	0
		101	1
		110	1
		111	0

5. No contexto de projeto de um sistema digital, "instanciar um componente" significa:

- a) defini-lo e adicioná-lo a uma biblioteca
- b) dotá-lo de uma interface standard com clock e reset
- c) utilizá-lo, ligá-lo e eventualmente parametrizá-lo
- d) proceder à sua simulação, aplicando diretamente estímulos às suas entradas

6. A construção de VHDL "wait for...":

- a) serve para sintetizar uma unidade de atraso programável
- b) é suportada para simulação e síntese
- c) serve para sintetizar uma unidade de atraso programável e é suportada quer para simulação quer para síntese
- d) é suportada apenas para simulação

7. No desenvolvimento de código VHDL, a escolha dos sinais que integram a lista de sensibilidade dos processos

- a) é irrelevante para efeitos de simulação
- b) não afeta a simulação nem a síntese de circuitos combinacionais
- c) pode contribuir para minimizar o tempo despendido em simulação
- d) pode contribuir para minimizar a utilização dos recursos da FPGA

8. O fluxo de projeto de sistemas digitais baseados em FPGA consiste na seguinte sequência ordenada de operações:

- a) programação do dispositivo, teste e depuração, modelação, implementação, síntese, simulação
- b) teste e depuração, simulação, modelação, implementação, síntese, programação do dispositivo,
- c) modelação, simulação, síntese, implementação, programação do dispositivo, teste e depuração
- d) implementação, programação do dispositivo, modelação, simulação, síntese, teste e depuração

9. A entity que forma uma testbench em VHDL:

- a) só pode ter portos de entrada
- b) só pode ter portos de saída
- c) tem de ter portos de entrada e de saída
- d) não possui normalmente portos

10. O corpo de uma *architecture* em VHDL pode conter:

- a) só instanciação e interligação de componentes
- b) só atribuições concorrentes e/ou condicionais
- c) só processos para modelar atividades concorrentes
- d) todas as restantes respostas estão corretas

11. A *package* NUMERIC_STD da biblioteca IEEE de VHDL define entre outras coisas:

- a) operadores para realizar a concatenação e *slicing* de sinais e portos do tipo `std_logic_vector`
- b) templates de código reutilizáveis para construção de unidades aritméticas e lógicas baseadas no tipo `std_logic_vector`
- c) operadores relacionais (e.g. `>` e `<=`) utilizáveis com o tipo de dados `std_logic_vector`
- d) os tipos de dados `signed`, `unsigned` e respetivos operadores aritméticos

12. A gama de representação de um sinal declarado como `signed(9 downto 0)` é:

- a) 0 a +1023
- b) -512 a +511
- c) -511 a +511
- d) -512 a +512

13. Numa memória RAM de dois portos, em que barramentos de endereço são de 4 bits e os de dados são de 32 bits, o número total de bits de dados armazenados é:

- a) 512 bits
- b) 128 bits
- c) 256 bits
- d) 512 bits

14. Considerando as declarações na figura ao lado, se $A = "0101"$ e $B = "0010"$ o resultado da operação é:

```
signal A, B : unsigned(3 downto 0);
signal Q, R : unsigned(3 downto 0);

...
Q <= A / B;
R <= A rem B;
```

- a) $Q = "0010"$ e $R = "0001"$
- b) $Q = "0010"$ e $R = "0010"$
- c) $Q = "0011"$ e $R = "0001"$
- d) $Q = "1011"$ e $R = "1001"$

15. Uma memória RAM com um porto permite:

- a) Escrever em dois endereços da memória ao mesmo tempo
- b) Ler de dois endereços da memória ao mesmo tempo
- c) Escrever e ler em diferentes endereços da memória ao mesmo tempo
- d) Escrever e ler ao mesmo tempo mas apenas no mesmo endereço de memória

16. Uma memória RAM:

- a) é uma memória apenas de leitura
- b) é principalmente usada para armazenamento de informação de forma permanente/não volátil
- c) garante que num dado instante pode ser lida qualquer posição de memória
- d) possui todas as características enumeradas nas alíneas restantes

17. Um engenheiro de sistemas digitais pouco familiarizado com a linguagem VHDL escreveu 4 alternativas diferentes de um flip-flop D com reset assíncrono. Qual delas é a correta?

a) process(rst, clk) begin if (rst='1') then dataOut <= '0'; elsif (clk='1') then dataOut <= dataIn; end if; end process;	b) process(rst, clk) begin if (rst='1') then dataOut <= '0'; elsif rising_edge(clk) then dataOut <= dataIn; end if; end process;	c) process(clk) begin if rising_edge(clk) then if (rst='1') then dataOut <= '0'; else dataOut <= dataIn; end if; end if; end process;	d) process(rst) begin if (rst='1') then dataOut <= '0'; else dataOut <= dataIn; end if; end process;
---	--	---	--

18. Considere o seguinte trecho de código VHDL, relativo a um comparador com duas saídas (geSigned e geUnsigned):

```
port(a, b : in std_logic_vector (3 downto 0);
...
geSigned <= '1' when (signed(a) >= signed(b)) else '0';
geUnsigned <= '1' when (unsigned(a) >= unsigned(b)) else '0';
```

Assumindo que $a = "1111"$ e $b = "1011"$, as saídas vão possuir, respetivamente, os seguintes valores:

- a) geSigned <= '0' e geUnsigned <= '1'
- b) geSigned <= '0' e geUnsigned <= '0'
- c) geSigned <= '1' e geUnsigned <= '1'
- d) geSigned <= '1' e geUnsigned <= '0'

19. As construções for ... generate

- a) pode ser escrita dentro de um processo
- b) deve ser escrita no corpo de uma arquitetura e dentro de um processo
- c) deve ser escrita no corpo de uma arquitetura
- d) nenhuma das opções anteriores está correta

20. Um Estudante precisa de usar um divisor de frequência no circuito que está a implementar. No entanto para o fazer apenas tem à sua disposição um simples contador módulo 16. Considerando que a frequência original precisa de ser dividida por 8, qual dos bits de saída do contador o estudante deve usar

- a) O bit 0
- b) O bit 2
- c) O bit 1
- d) Não é possível usar o contador para efetuar essa divisão

21. A síntese do seguinte código resulta em qual dos seguintes componentes:

```
with DataIn select
DataOut <= "00000001" when "000", "00000010" when "001", "00000100" when "010", "00001001" when
"011", "00010000" when "100", "00100000" when "101", "01000000" when "110", "10000000" when "111",
"00000000" when others;
```

- a) um codificador de 3:8
- b) um decodificador de 3:8
- c) um decodificador de 8:3
- d) nenhuma das respostas restantes

22. Considerando as declarações:

```
type my_type is ('U', 'X', 'O', '1', 'Z', 'W', 'L', 'H', '-');
```

qual será o resultado da expressão `my_type 'pos('W')`

- a) 0
- b) 6
- c) 5
- d) Nenhuma das anteriores

23. Considere o código VHDL seguinte:

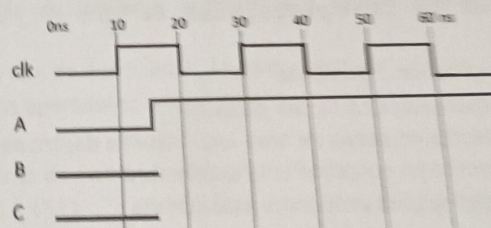
```
entity ALU is
  port (op : in std_logic_vector(1 downto 0);
        A, B : in std_logic_vector(3 downto 0);
        R : out std_logic_vector(3 downto 0));
end ALU;
architecture Behavioral of ALU is
begin
  R <= std_logic_vector(signed(A) - signed(B)) when op = "00" else
        std_logic_vector(signed(A) + signed(B)) when op = "01" else
        A xor B when op = "10" else
        A or B;
end Behavioral;
```

Quando `A="0100"`, `B="1011"` e `op="00"`, o sinal `R` toma o valor:

- a) "1001"
- b) "1111"
- c) "0011"
- d) "1011"

24. O Considere o seguinte excerto de código VHDL. Assumindo que `clk` e `A` evoluem de acordo com o diagrama temporal dado, o sinal `C`:

```
process (clk)
begin
  if (rising_edge(clk)) then
    B <= A;
    C <= B;
  end if;
end process
```



- a) muda de '0' para '1' aos 15 ns
- b) muda de '0' para '1' aos 50 ns
- c) muda de '0' para '1' aos 30 ns
- d) mantém sempre o valor '0'

25. Considere o módulo em VHDL descrito ao lado, instanciado num sistema com $K=4$ e em que a frequência do sinal de relógio de entrada ($clkIn$) é 200 MHz. Nestas circunstâncias a frequência do sinal de relógio de saída ($clkOut$) vai ser:

- a) 20 MHz
- b) 25 MHz
- c) 50 MHz
- d) 40 MHz

```
entity FreqDivider is
    generic(K : positive);
    port(clkIn : in std_logic;
         clkOut : out std_logic);
end FreqDivider;

architecture Behavioral of FreqDivider is
    signal s_counter : natural := 0;
    signal s_clk : std_logic := '0';
begin
    process(clkIn)
    begin
        if (rising_edge(clkIn)) then
            if (s_counter >= K) then
                s_counter <= 0;
                s_clk <= not s_clk;
            else
                s_counter <= s_counter + 1;
            end if;
        end if;
    end process;
    clkOut <= s_clk;
end Behavioral;
```

26. Para o código da questão anterior, o *duty-cycle* do sinal de saída ($clkOut$) vai ser:

- a) sempre igual ou superior a 50% e dependente de K
- b) sempre 50% independentemente do *duty-cycle* do sinal de entrada ($clkIn$)
- c) sempre igual ou inferior a 50% e dependente de K
- d) 50% se o *duty-cycle* do sinal de entrada ($clkIn$) também for 50%

27. Qual dos seguintes trechos de código corresponde a um processo que implementa um contador binário natural, com entrada de *reset* e entrada de ativação de carregamento paralelo síncronos?

a)

```
process(clk)
begin
    if (rising_edge(clk)) then
        if (reset = '1') then
            s_cntValue <= (others => '0');
        elsif (loadEn = '1') then
            s_cntValue <= dataIn;
        else
            s_cntValue <= s_cntValue + 1;
        end if;
    end if;
end process;
```

c)

```
process(reset, loadEn, clk)
begin
    if (reset = '1') then
        s_cntValue <= (others => '0');
    elsif (loadEn = '1') then
        s_cntValue <= dataIn;
    elsif (rising_edge(clk)) then
        s_cntValue <= s_cntValue + 1;
    end if;
end process;
```

b)

```
process(reset, clk)
begin
    if (reset = '1') then
        s_cntValue <= (others => '0');
    elsif (rising_edge(clk)) then
        if (loadEn = '1') then
            s_cntValue <= dataIn;
        else
            s_cntValue <= s_cntValue + 1;
        end if;
    end if;
end process;
```

d)

```
process(loadEn, clk)
begin
    if (loadEn = '1') then
        s_cntValue <= dataIn;
    elsif (rising_edge(clk)) then
        if (reset = '1') then
            s_cntValue <= (others => '0');
        else
            s_cntValue <= s_cntValue + 1;
        end if;
    end if;
end process;
```


28. Analise o seguinte módulo escrito em VHDL:

```
entity Circuito is
    generic (K : positive := 10);
    port (clk : in std_logic;
          reset : in std_logic;
          start : in std_logic;
          timerOut : out std_logic);
end Circuito;

architecture Behavioral of Circuito is
    signal a_count : integer := 0;
begin
    process (clk)
    begin
        if (rising_edge(clk)) then
            if (reset = '1') then timerOut <= '0'; a_count <= 0;
            elsif (a_count = 0) then
                if (start = '1') then
                    timerOut <= '1';
                    a_count <= a_count + 1;
                else timerOut <= '0';
                end if;
            else
                if (a_count = K) then
                    timerOut <= '0';
                    a_count <= 0;
                else
                    timerOut <= '1';
                    a_count <= a_count + 1;
                end if;
            end if;
        end if;
    end process;
end Behavioral;
```

O código apresentado descreve um:

- contador binário *free running* de módulo "K"
- módulo que ativa a saída durante "K" ciclos de relógio após a detecção da entrada *start* com o valor lógico '1'
- módulo que ativa a saída durante "K" ciclos de relógio após a desativação (transição para "0") da entrada *start*
- módulo que ativa a saída se a entrada *start* tiver uma duração igual a "K" ciclos de relógio

29. Considere uma memória RAM com a seguinte interface:

```
entity RAM is
    generic (addrBusSize : positive := 3;
            dataBusSize : positive := 8);
    port (clk : in std_logic;
          writeEnable : in std_logic;
          writeAddress : in std_logic_vector((addrBusSize - 1) downto 0);
          writeData : in std_logic_vector((dataBusSize - 1) downto 0);
          readAddress : in std_logic_vector((addrBusSize - 1) downto 0);
          readData : out std_logic_vector((dataBusSize - 1) downto 0));
end RAM;
```

A esta memória aplica-se a afirmação seguinte:

- é uma memória de um porto
- é uma memória de *dataBusSize* portos
- a leitura é realizada de modo síncrono
- é impossível determinar se a leitura é síncrona ou assíncrona

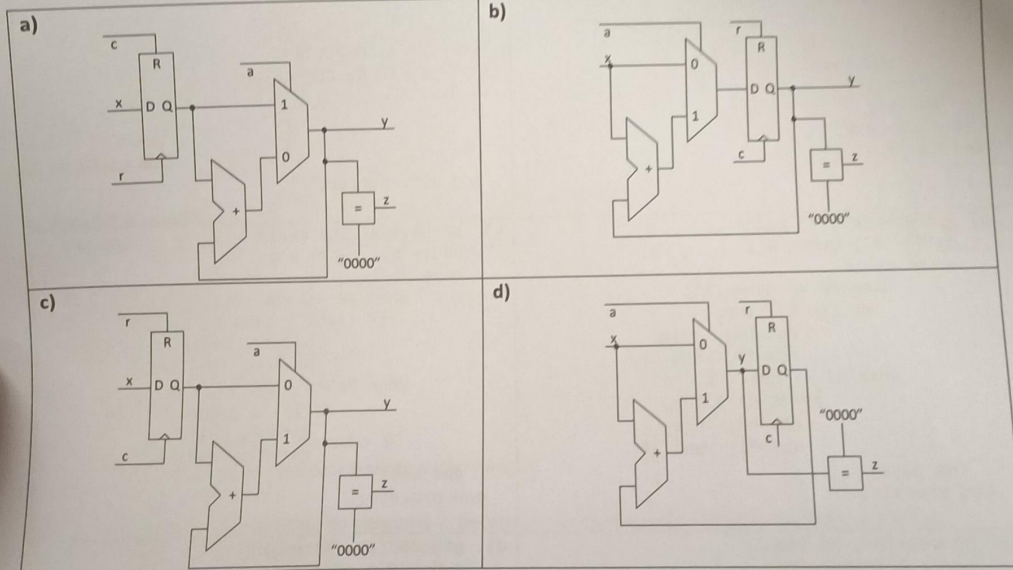
30. Ainda referente à pergunta anterior, para instanciar uma memória que armazena 64K palavras de 32 bits, a respetiva construção **generic map** deve ser parametrizada da seguinte forma:

- a) addrBusSize => 16, dataBusSize => 32
- b) addrBusSize => 16, dataBusSize => 64000
- c) addrBusSize => 64000, dataBusSize => 32
- d) addrBusSize => 64, dataBusSize => 64

31. Considerando que x , c , a e z são sinais de um bit e que x e y são vetores de 4 bits, a síntese do seguinte trecho de código VHDL resulta no circuito lógico:
Nota: Assuma que o reset nas figuras é síncrono.

```
process(c)
begin
  if (rising_edge(c)) then
    if (x = '1') then
      y <= "0000";
    elsif (a = '1') then
      y <= y + x;
    else
      y <= x;
    end if;
  end if;
end process;

z <= '1' when (y = "0000") else '0';
```



32. O código VHDL à esquerda na tabela descreve uma MEF. O código à direita descreverá uma MEF funcionalmente equivalente se for completado, na zona indicada, com:

<pre> library IEEE; use IEEE.STD_LOGIC_1164.all; entity DetPar is port(clk, reset, X : in std_logic; odd, evn : out std_logic); end DetPar; architecture Given of DetPar is type TState is (PP, PI); signal st : TState; begin process(reset, clk) begin if (reset='1') then st<= PP; odd<='0'; evn<='1'; else if (rising_edge(clk)) then case st is when PP => if (X='1') then st<=PI; odd<='1'; evn<='0'; end if; when PI => if (X='1') then st<=PP; odd<='0'; evn<='1'; end if; end case; end if; end process; end Given; </pre>	<pre> library IEEE; use IEEE.STD_LOGIC_1164.all; entity DetPar is port(clk, reset, X : in std_logic; odd, evn : out std_logic); end DetPar; architecture Alternative of DetPar is type TState is (PP, PI); signal ps, ns : TState; begin process(clk, reset) begin if (reset='1') then ps<=PP; else if (rising_edge(clk)) then ps<=ns; end if; end if; end process; ----- ----- A COMPLETAR ----- TO BE COMPLETED ----- end Alternative; </pre>
<pre> a) process(ps, X) begin odd<='0'; evn<='0'; case ps is when PP => evn<='1'; if (X='1') then ns<=PI; else ns<=PP; end if; when PI => odd<='1'; if (X='1') then ns<=PP; else ns<=PI; end if; end case; end process; </pre>	<pre> b) process(ps, X) begin odd<='0'; evn<='0'; case ns is when PP => evn<='1'; if (X='1') then ns<=PI; else ns<=PP; end if; when PI => odd<='1'; if (X='1') then ns<=PP; else ns<=PI; end if; end case; end process; </pre>
<pre> c) process(ps, X) begin odd<='0'; evn<='0'; case ps is when PP => if (X='1') then ns<=PI; odd<='1'; else ns<=PP; evn<='1'; end if; when PI => if (X='1') then ns<=PP; evn<='1'; else ns<=PI; odd<='1'; end if; end case; end process; </pre>	<pre> d) process(ps, X) begin evn<='0'; odd<='0'; case ps is when PP => odd<='1'; if (X='1') then ns<=PI; else ns<=PP; end if; when PI => evn<='1'; if (X='1') then ns<=PP; else ns<=PI; end if; end case; end process; </pre>